

Librerías Implementadas

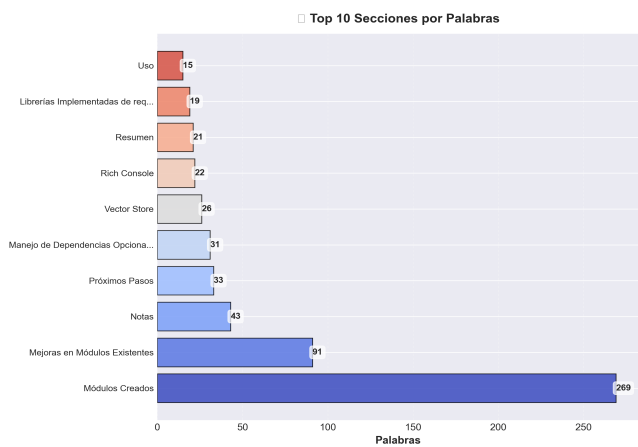
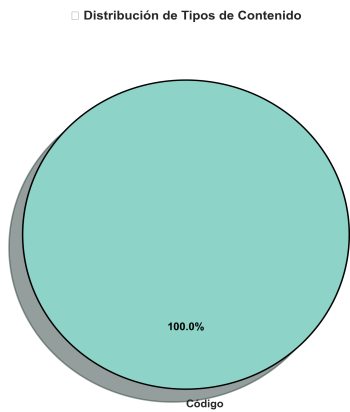
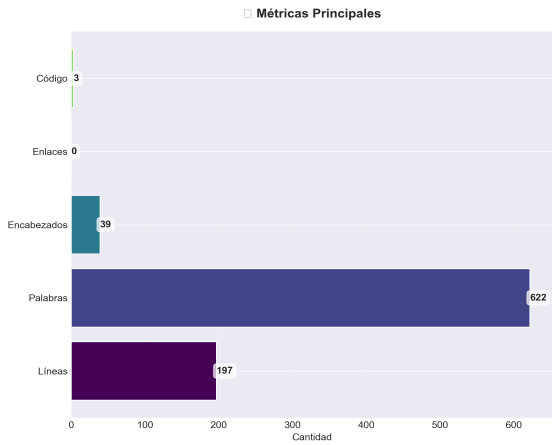
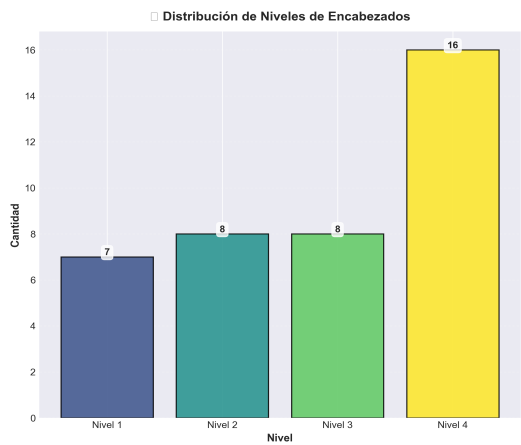
Análisis y Documentación Completa

Generado el: 25 de November de 2025 a las 11:59

Análisis Visual Completo

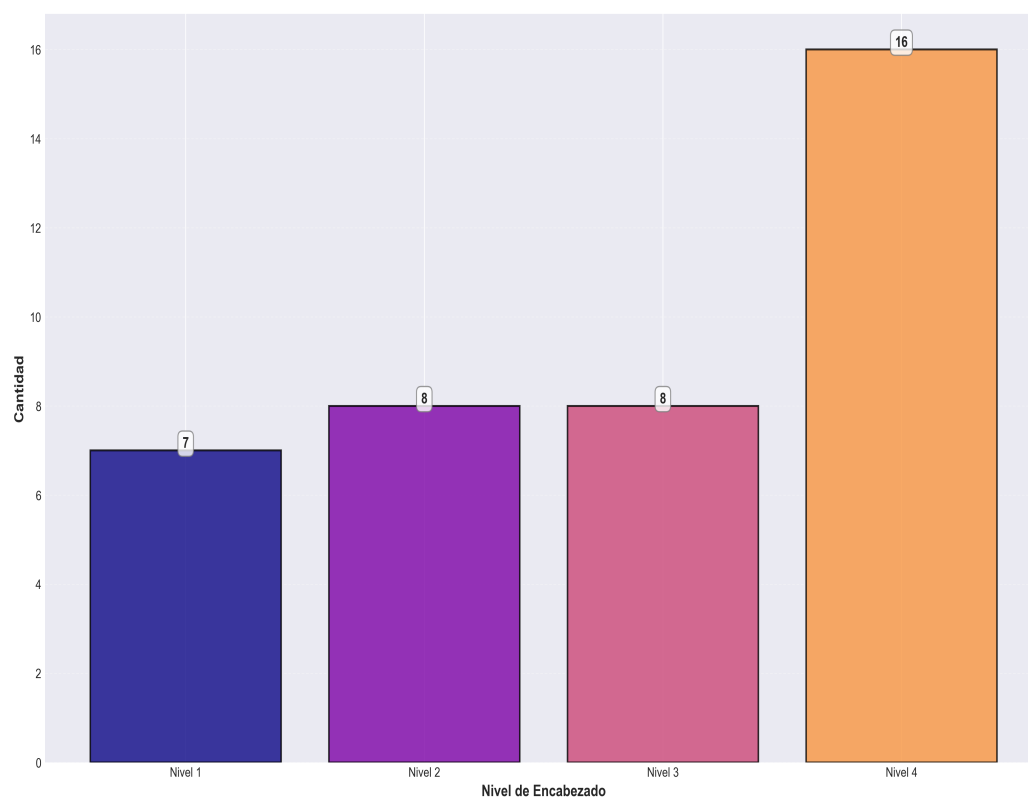
Dashboard Principal

□
 Análisis Completo: LIBRERIAS_IMPLEMENTADAS

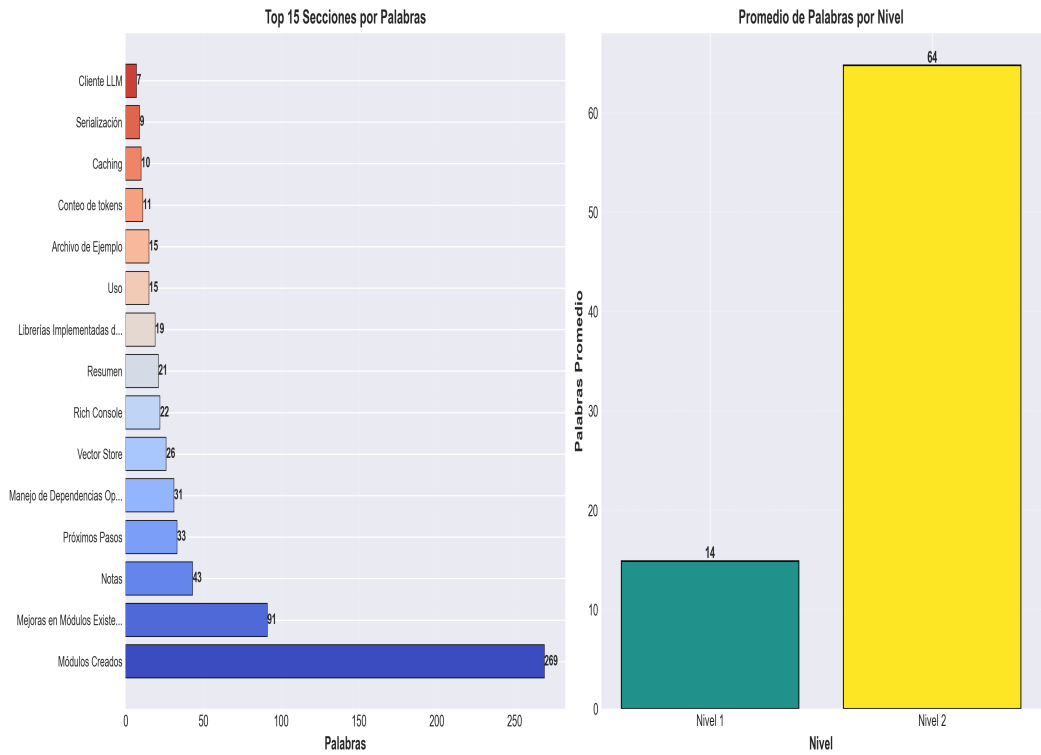


Análisis Adicionales

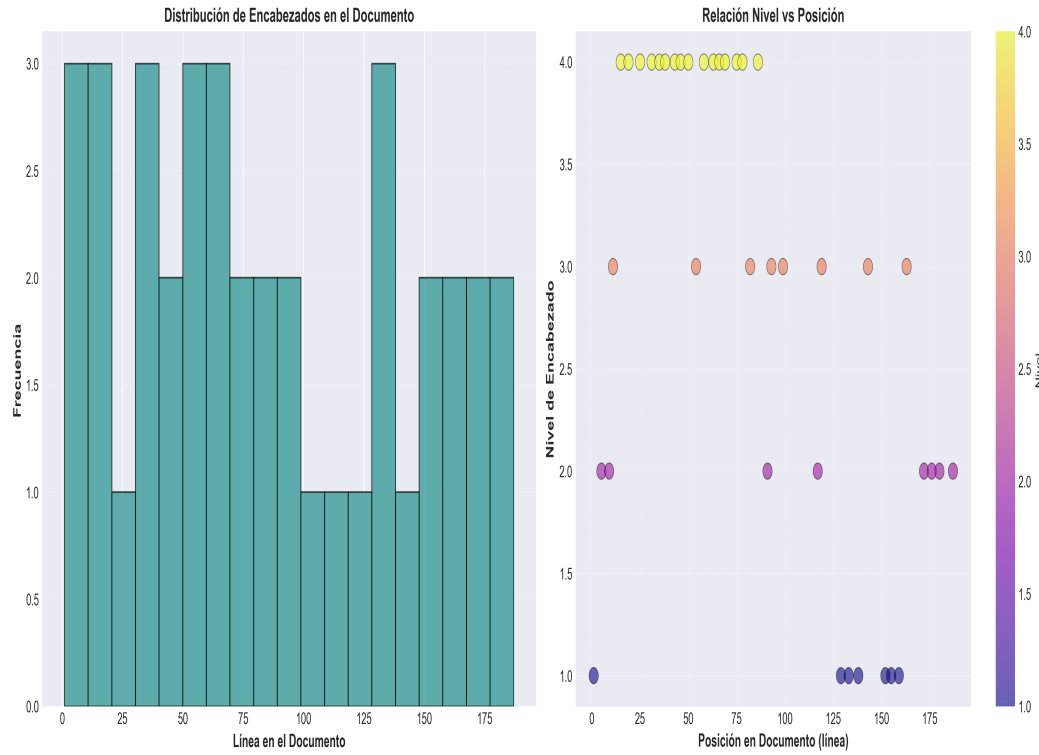
□ Estructura de Encabezados: LIBRERIAS_IMPLEMENTADAS



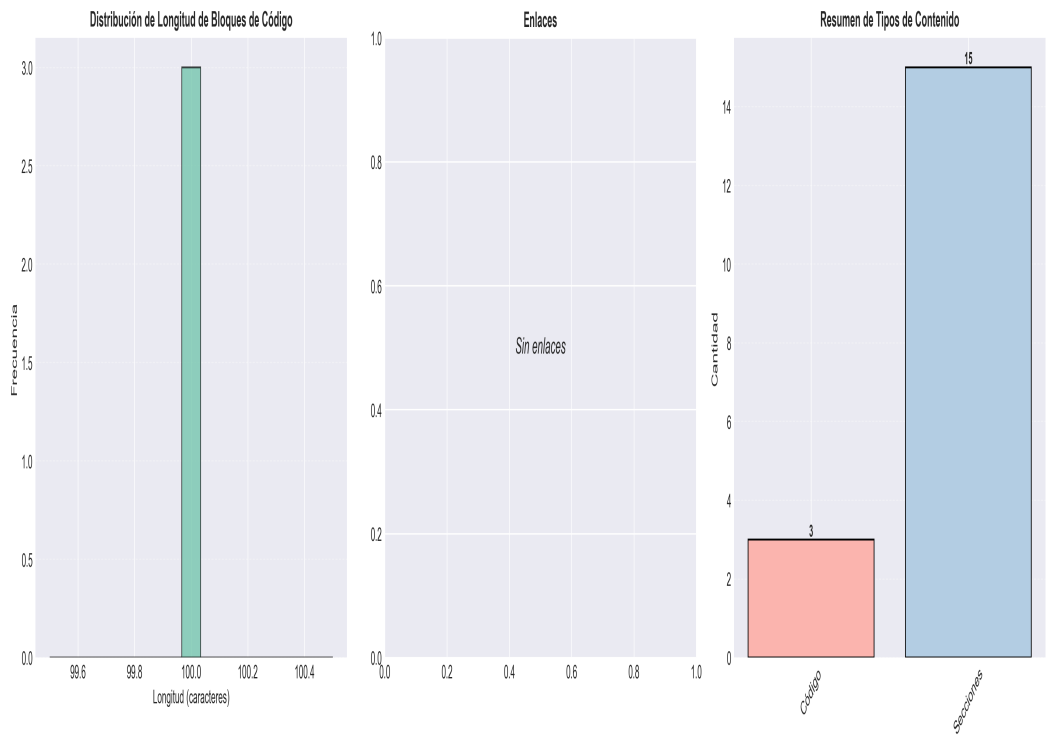
□ Análisis de Complejidad: LIBRERIAS_IMPLEMENTADAS



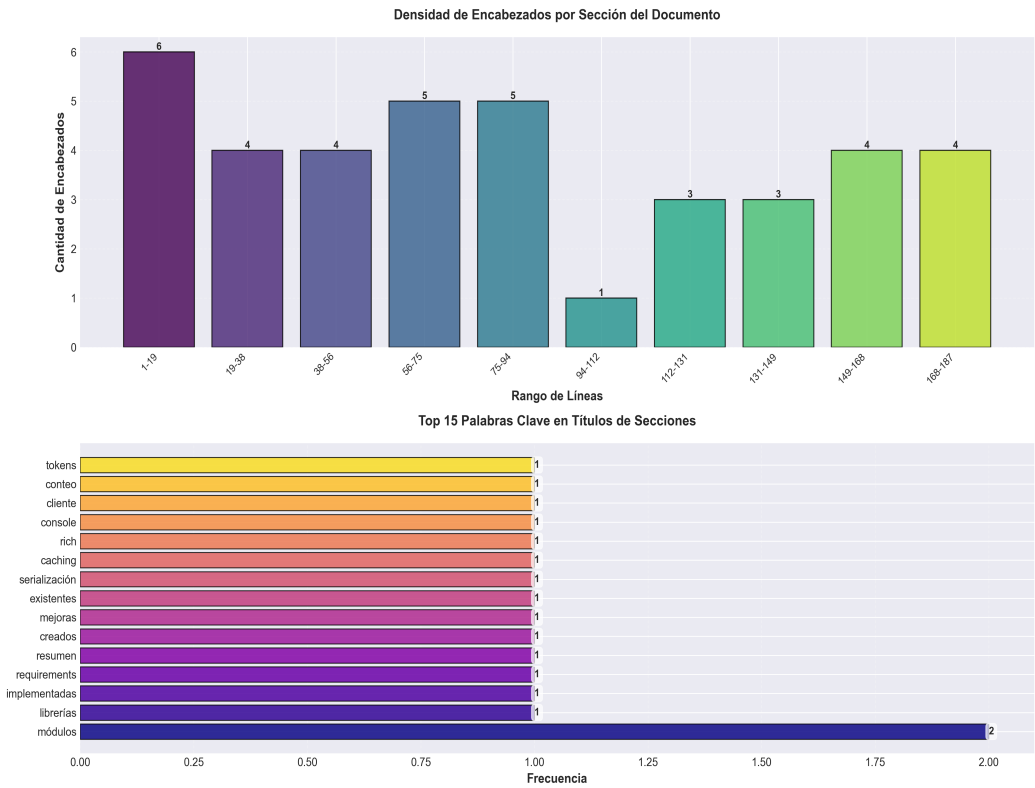
📄 Análisis Temporal y Distribución: LIBRERIAS_IMPLEMENTADAS



▣ Análisis Detallado de Contenido: LIBRERIAS_IMPLEMENTADAS



■
 Análisis de Densidad y Frecuencia: LIBRERIAS_IMPLEMENTADAS



Resumen Ejecutivo: Este documento contiene 622 palabras distribuidas en 197 líneas, con 39 encabezados organizados en 15 secciones principales.

Estadísticas del Documento

Métrica	Valor
Total de líneas	197
Total de palabras	622
Total de caracteres	5,110
Encabezados	39
Bloques de código	3
Tablas	0
Enlaces	0
Imágenes	0
Secciones principales	15

Índice de Secciones

Nivel	Título	Línea
1	Librerías Implementadas de requirements.txt	1
2	Resumen	5
2	Módulos Creados	9
2	Mejoras en Módulos Existentes	91
2	Uso	117
1	Serialización	129
1	Caching	133
1	Rich Console	138
1	Cliente LLM	152
1	Conteo de tokens	155
1	Vector Store	159
2	Archivo de Ejemplo	172
2	Manejo de Dependencias Opcionales	176
2	Notas	180
2	Próximos Pasos	187

Resumen del Contenido

Librerías Implementadas de requirements.txt

Este documento describe las librerías de `requirements.txt` que han sido implementadas en el código.

Resumen

Se han creado módulos avanzados que integran las principales librerías de `requirements.txt` con manejo de errores y fallbacks apropiados.

Módulos Creados

1. `core/advanced_utils.py`

Módulo principal que integra múltiples librerías:

- `orjson`: Serialización JSON de alto rendimiento
- `msgpack`: Serialización binaria eficiente
- `cachetools`: Caché en memoria con TTL, LRU, LFU
- `redis`: Caché distribuido con Redis
- `diskcache`: Caché persistente en disco
- `aiocache`: Caché asíncrono
- `hydra-core`: Gestión avanzada de configuración
- `omegaconf`: Configuración estructurada
- `pydantic-settings`: Configuración con validación
- `dynaconf`: Configuración dinámica
- `structlog`: Logging estructurado
- `python-json-logger`: Logging en formato JSON
- `rich`: Consola enriquecida con tablas, paneles, progreso
- `prometheus-client`: Métricas de Prometheus
- `psutil`: Métricas del sistema
- `memory-profiler`: Profiling de memoria
- `joblib`: Procesamiento paralelo eficiente

- **jsonschema**: Validación con esquemas JSON
- **cerberus**: Validación de datos
- **arrow**: Manipulación de fechas
- **python-dateutil**: Utilidades de fecha

`2. `core/llm_advanced.py``

Módulo para LLMs y RAG:

- **langchain**: Framework para aplicaciones LLM
- **langchain-community**: Extensiones de LangChain
- **llama-index**: Framework de indexación para LLM
- **tiktoken**: Conteo preciso de tokens
- **sentence-transformers**: Modelos de embeddings
- **chromadb**: Base de datos vectorial
- **pinecone-client**: Base de datos vectorial en la nube
- **weaviate-client**: Base de datos vectorial
- **qdrant-client**: Base de datos vectorial
- **faiss-cpu**: Búsqueda de similitud eficiente
- **guidance**: Framework para prompt engineering
- **outlines**: Generación estructurada

`3. `core/visualization_advanced.py``

Módulo de visualización:

- **matplotlib**: Gráficos 2D
- **seaborn**: Visualización estadística
- **plotly**: Gráficos interactivos

Mejoras en Módulos Existentes

``core/monitoring.py``

Se ha mejorado para integrar:

- **prometheus-client**: Métricas de Prometheus opcionales
- **psutil**: Métricas del sistema mejoradas

``core/utils.py``

[Nota: El documento original tiene 197 líneas. Se muestra un resumen de las primeras 100 líneas.]